
CORSO DI ARCHITETTURE DEGLI ELABORATORI A.A. 2025/2026
PROF.SSA VALENTINA CASOLA

ARCHITETTURA DEL CALCOLATORE

Dr. Alessandra Somma
alessandra.somma@unina.it

INTRODUZIONE AI SISTEMI DI ELABORAZIONE

- Un **sistema** è definito come una pluralità di elementi collegati tra loro per eseguire una assegnata operazione.
 - La *funzione* svolta da un sistema è ciò che esso deve fare così come formalizzato dalla sua specifica.
 - Il *comportamento* del sistema è ciò che esso fa per implementare la sua funzione.
 - La *struttura* è l'insieme di ciò che gli permette di generare il suo comportamento.
 - Il *servizio* erogato dal sistema è il suo comportamento così come percepito da un suo utente, che può essere un umano o un altro sistema.
- Al fine di erogare i propri servizi, un sistema è composto da sottosistemi dotati di capacità elaborativa (**processing element**) e sottosistemi demandati alla comunicazioni (**sottosistemi di interconnessione**), ossia alla trasmissione delle informazioni sia all'interno sia all'esterno del sistema stesso.
 - Sia i gli elementi di processazione sia i sottosistemi di interconnessione sono realizzati da un insieme di componenti o **blocchi funzionali**, ciascuno dedicato a svolgere un determinato insieme di funzioni.

INTRODUZIONE AI SISTEMI DI ELABORAZIONE

- Un **sistema di elaborazione** è una macchina in grado di eseguire operazioni (**istruzioni**) interagendo con il mondo esterno al fine di ricevere informazioni in ingresso (**dati di ingresso**) e fornire in uscita i relativi risultati (**dati di uscita**).
- Un sistema di elaborazione è dato da una parte detta **hardware**, che indica la struttura fisica e tangibile della macchina, ed una parte intangibile detta **software** che rappresenta l'insieme di istruzioni, programmi e dati che indicano all'hardware cosa fare.
- Ad una data sequenza di istruzioni finalizzata all'esecuzione di una funzione, viene dato il nome di **programma**. Ciascun programma è scritto in un particolare linguaggio, detto **linguaggio di programmazione**.
- Il programma è la descrizione formalizzata di un **algoritmo** mediante un linguaggio di programmazione.

CLASSIFICAZIONE DEI SISTEMI

- In ambito informatico, sono state proposte varie classificazioni dei sistemi di elaborazione rispetto a diversi parametri. Ad esempio, è possibile classificarli in base a:

1. Codifica dell'informazione

- Sistemi *Digitali*, in cui le informazioni sono memorizzate, elaborate e trasmesse in forma digitale, ovvero insieme di valori discreti
- Sistemi *Analogici* in cui l'informazione viene rappresentata utilizzando intervalli di valori continui

2. Comportamento delle uscite rispetto agli ingressi

- *Combinatori*, in cui i valori presenti sulle uscite principali dipendono esclusivamente dai valori presenti sugli ingressi principali

Una **macchina combinatoria** è la macchina per l'elaborazione più semplice di tutte: risolvere problemi con una macchina combinatoria comporta **enumerare in modo esplicito tutte le possibili configurazioni** di ingresso e indicare in corrispondenza il valore di uscita. Un dispositivo puramente combinatorio **non è adatto** a risolvere problemi che richiedono una memoria interna.

CLASSIFICAZIONE DEI SISTEMI

- In ambito informatico, sono state proposte varie classificazioni dei sistemi di elaborazione rispetto a diversi parametri. Ad esempio, è possibile classificarli in base a:

1. Codifica dell'informazione

- Sistemi *Digitali*, in cui le informazioni sono memorizzate, elaborate e trasmesse in forma digitale, ovvero insieme di valori discreti
- Sistemi *Analogici* in cui l'informazione viene rappresentata utilizzando intervalli di valori continui

2. Comportamento delle uscite rispetto agli ingressi

- *Combinatori*, in cui i valori presenti sulle uscite principali dipendono esclusivamente dai valori presenti sugli ingressi principali
- *Sequenziali*, in cui i valori presenti sulle uscite dipendono dai valori presenti sugli ingressi non solo in quell'istante ma anche in tutti quelli precedenti

Gli **automi** sono una prima astrazione di macchina dotata di **memoria** che esegue algoritmi. Introducono il concetto fondamentale di **stato** che informalmente può essere definito come una particolare condizione della macchina, in conseguenza della quale la macchina reagisce con una determinata uscita ad un determinato ingresso. Sono però dispositivi *a memoria finita*, con **potere espressivo limitato**, quindi inadatti a risolvere quei problemi che non consentono di limitare a priori la lunghezza delle sequenze di ingresso di cui tenere memoria.

CLASSIFICAZIONE DEI SISTEMI

- In ambito informatico, sono state proposte varie classificazioni dei sistemi di elaborazione rispetto a diversi parametri. Ad esempio, è possibile classificarli in base a:

3. Modello Architeturale

- SISD - Single Instruction Stream, Single Data Stream
- SIMD – Single Instruction Stream, Multiple Data Stream
- MISD - Multiple Instruction Stream, Single Data Stream
- MIMD - Multiple Instruction Stream, Multiple Data Stream

CLASSIFICAZIONE DEI SISTEMI

- In ambito informatico, sono state proposte varie classificazioni dei sistemi di elaborazione rispetto a diversi parametri. Ad esempio, è possibile classificarli in base a:

4. Funzione svolta dal singolo nodo di elaborazione

- General Purpose, in grado di eseguire istruzioni che consentono di elaborare programmi di qualsiasi tipologia;
- Special Purpose, realizzati per eseguire ad elevate prestazioni, uno o più programmi aventi caratteristiche preassegnate;
- DSP (Digital Signal Processor), tipo special purpose dedicate all'elaborazione di segnali digitali attraverso elaborazione di funzioni matematiche particolarmente complesse (FFT, DFFT, etc.);
- GPU (Graphic Processing Unit), tipo special purpose dedicate alla elaborazione di funzioni per la grafica;
- GPGPU (General Purpose GPU), estensione e generalizzazione delle GPU utilizzabile non solo come acceleratore grafico, ma come sistema di uso generale ad elevate prestazioni.

ALGORITMO

- L'algoritmo è una **sequenza finita** di azioni elaborative (o di passi di elaborazione) che risolve **automaticamente** un problema. Si può esprimere come:

$$Y=F(X)$$

dove X sono i *dati di ingresso*, Y i *dati di uscita* ed F è il procedimento costituito dall'insieme di azioni elaborative che vanno eseguite secondo un definito ordine.

Sequenza vuol dire che le azioni elaborative vengono eseguite una alla volta secondo un ordine stabilito

Finita è inteso con riferimento allo spazio (numero finito di azioni elaborative) e al tempo (ogni azione elaborativa si realizza in un tempo finito)

Automaticamente vuol dire che esiste una macchina che può eseguirlo (esecutore) e che una volta avviata è in grado di evolvere da sola

- Il procedimento descritto dall'algoritmo è **deterministico**: la sequenza è rigidamente fissata e niente è lasciato al caso. Occorre definire:
 - L'insieme di tutte le possibili azioni elaborative
 - Una macchina in grado di eseguirle
 - Un linguaggio attraverso il quale descrivere ciascuna delle azioni e la sequenza di esse

RIEPILOGANDO:

- Un **algoritmo** viene descritto tramite un linguaggio
- Il linguaggio di descrizione di un algoritmo deve essere comprensibile all'**esecutore**, ovvero alla macchina automatica che lo esegue.
- Il linguaggio deve essere «non ambiguo», ovvero ciascuna frase costruita con esso deve evocare un'unica azione elaborativa.
- Un **programma** è la descrizione formalizzata di un algoritmo, espressa in un **linguaggio di programmazione**
- Un programma è una sequenza di istruzioni, ciascuna esprimente operazioni che l'esecutore può comprendere ed eseguire
- Il programma che deve essere eseguito è memorizzato nella **memoria dell'esecutore**

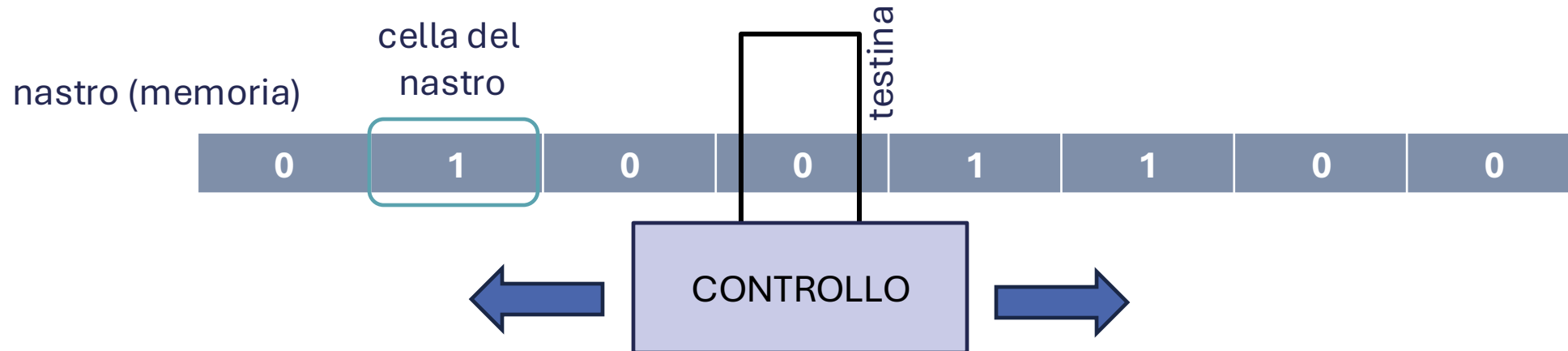
LA MACCHINA DI TURING

- Nel 1936, Alan Turing introdusse una **macchina astratta**, nota ad oggi con il nome di **Macchina di Turing** (MdT).
- È un modello fondamentale nella teoria dell'informatica.
- Ha consentito di ottenere importanti risultati relativamente alla **calcolabilità, alla complessità e all'equivalenza di algoritmi**.
- Consente di approfondire il concetto di algoritmo

La macchina di Turing è una **macchina ideale**, cioè indipendente da una sua possibile realizzazione fisica!

LA MACCHINA DI TURING

- La MdT è composta da un **nastro** (memoria), cioè una striscia continua di celle ciascuna delle quali può essere vuota o contenere i simboli di un alfabeto, idealmente infinito. Ha una **testina di lettura e scrittura**, che può operare su ogni singola cella del nastro, muovendosi a sinistra o a destra lungo il nastro per agire sulle altre celle.
- E' dotata di un **dispositivo di controllo** che in ogni istante si trova in uno stato (tra un numero finito di stati) e che per ciascuna coppia (stato, simbolo letto): cambia stato ed esegue una delle azioni elaborative



ELABORAZIONE CON LA MACCHINA DI TURING

- Parte da uno stato iniziale
- Sul nastro è scritto il valore dell'input
- Si legge il simbolo sotto la testina
- Sulla base di (input, stato) si ottiene: valore da scrivere sul nastro, nuovo stato, movimento left/right
- Si procede finché non si raggiunge uno stato finale
- Il nastro come luogo dove:
 - Trasformare l'input in output
 - Scrivere valori d'appoggio, risultati intermedi

MACCHINA DI TURING E ALGORITMO

- Una macchina di Turing che si arresti e trasformi un nastro t in uno t' rappresenta l'algoritmo per l'elaborazione $Y=F(X)$, ove X e Y sono rispettivamente codificati in t e t' .
- *Data una funzione $Y=F(X)$ esiste sempre una MdT in grado di calcolarla (tramite un processo automatico)?* La risposta è che **esistono funzioni non calcolabili**.
- **Tesi di Church:** non esiste alcun formalismo, per modellare una determinata computazione meccanica, che sia più potente della Macchina di Turing e dei formalismi ad essi equivalenti.
 - Nessuno è mai riuscito a dimostrare il contrario

Ogni algoritmo può essere codificato in termini di Macchina di Turing ed è quindi **ciò che può essere eseguito da una macchina di Turing**.

Un problema **non è risolubile** **algoritmicamente** se **nessuna** Macchina di Turing è in grado di fornire la soluzione al problema **in tempo finito**.

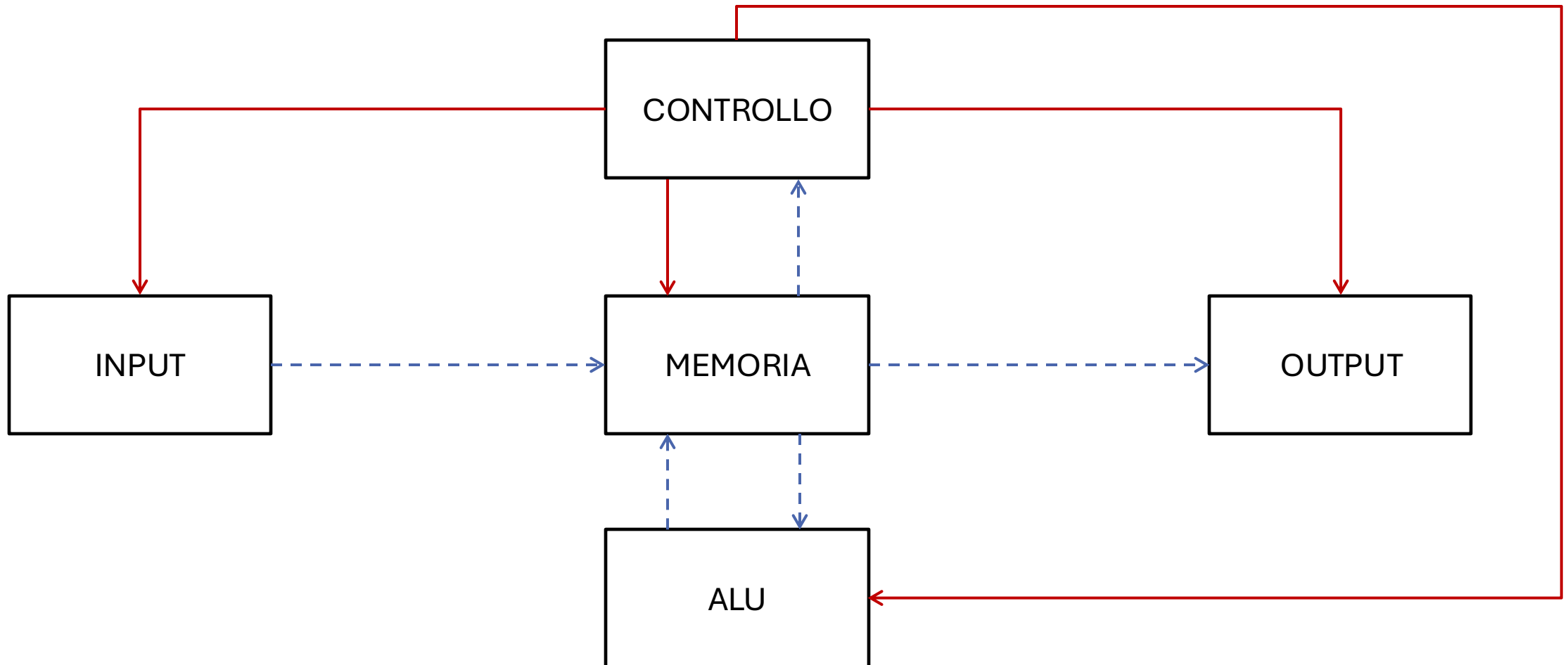
- Se esistono problemi che la MdT non può risolvere, esistono algoritmi che non possono essere calcolati.
- Sono detti problemi **decidibili** quei problemi che possono essere meccanicamente risolvibili da una macchina di Turing; sono indecidibili tutti gli altri.

DA ALAN TURING A JOHN VON NEUMANN

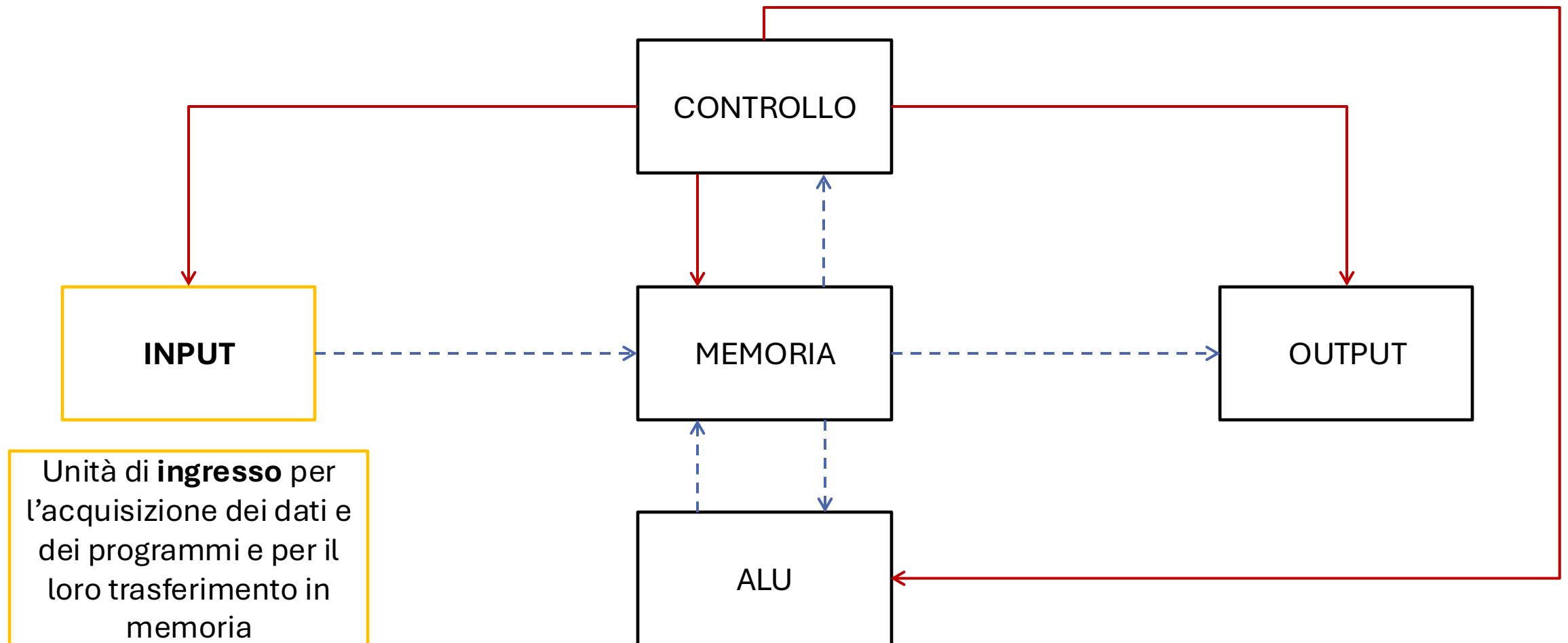
- Partendo dal progetto ENIAC insieme a Turing, Von Neumann definì il **modello di Von Neumann** che oggi contribuisce alla realizzazione di un calcolatore con un'architettura adottata ancora oggi nei moderni calcolatori.
- La macchina di Von Neumann (1945) fu modellata dalla Macchina di Turing Universale.
 - A differenza del nastro di Turing, la **memoria** è di dimensione **limitata**.
 - La macchina di Von Neumann, come modello di riferimento per sistemi non solo teorici, prevede anche la dimensione dell'interazione attraverso dispositivi di input ed output.

MODELLO DI VON NEUMANN

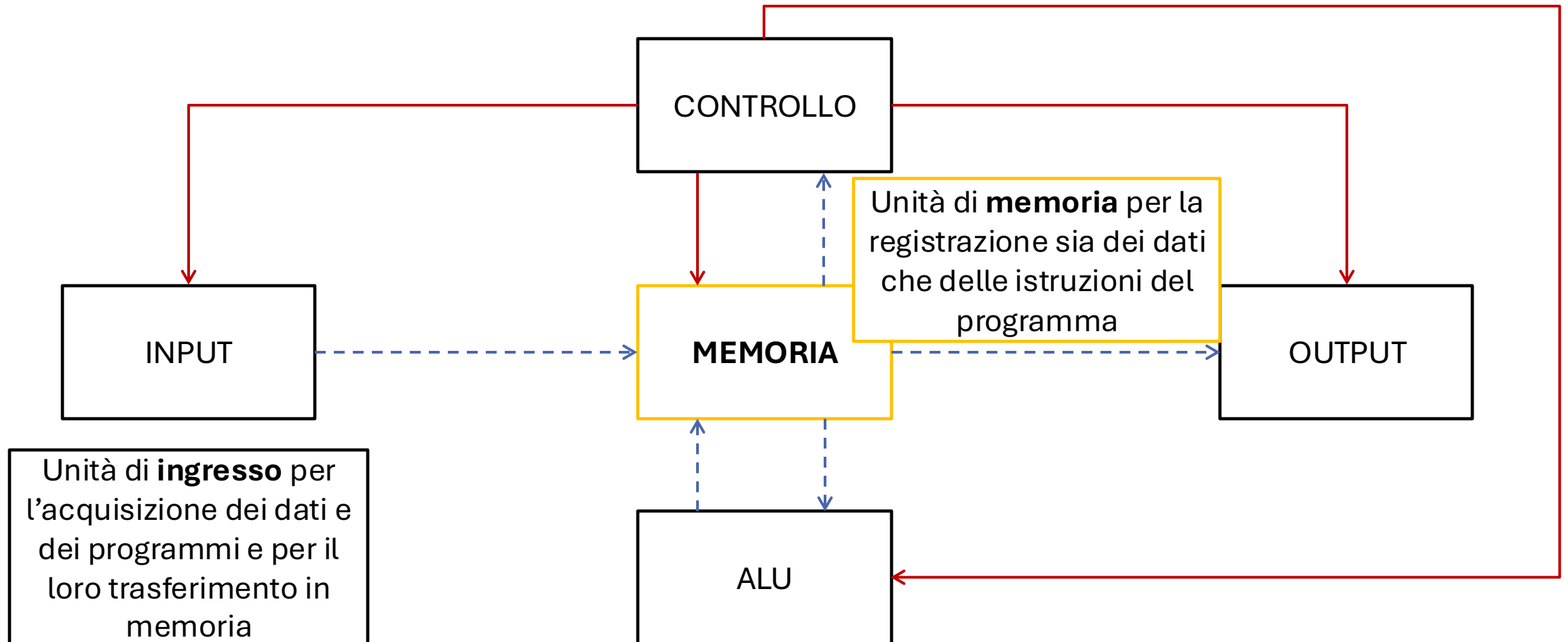
Le frecce rosse indicano il
flusso di controllo. Le frecce
blu il flusso dati.



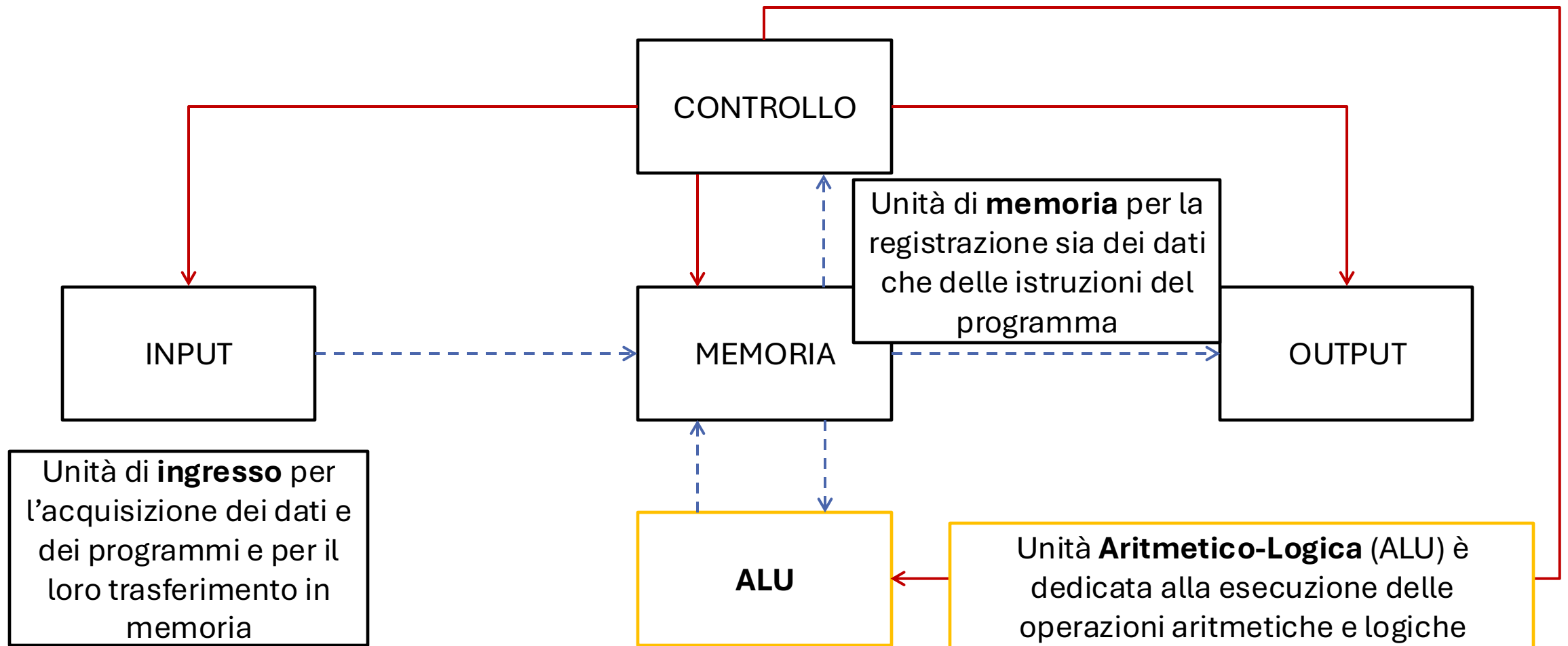
MODELLO DI VON NEUMANN



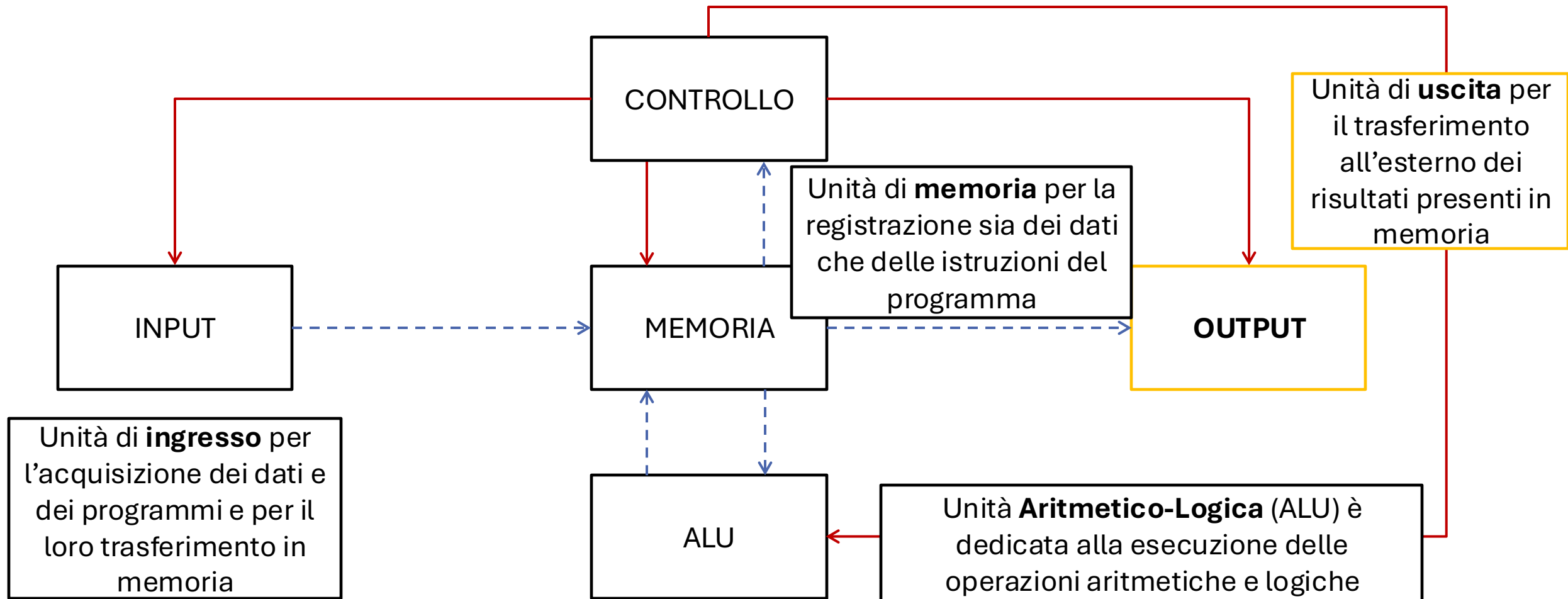
MODELLO DI VON NEUMANN



MODELLO DI VON NEUMANN



MODELLO DI VON NEUMANN



MODELLO DI VON NEUMANN: CARATTERISTICHE

- **L'unità di controllo** del modello di Von Neumann è la macchina che esegue il processo di elaborazione, ossia l'algoritmo che risolve un dato problema formalmente descritto mediante un programma.
- Il **programma** in esecuzione, costituito da una sequenza di istruzioni **è registrato nella memoria dell'elaboratore insieme ai dati** (programma memorizzato) su cui opera.
 - La macchina immagazzina nella propria memoria i dati su cui lavorare e le istruzioni per il suo funzionamento.
 - NOTA: esiste un modello differente, noto come architettura di Harvard, che utilizza una memoria separata e segnali di controllo diversi per dati ed istruzioni. Una sua implementazione modificata, che prevede la stessa memoria, ma due cache diverse per dati ed istruzioni, è oggi ampiamente utilizzata per motivi di efficienza.
- Le istruzioni vengono eseguite in maniera sequenziale.
- Rispetto alla MdT, il modello di VN ha **flessibilità operativa** → macchine nate per fare calcoli possono essere impiegate nella risoluzione di problemi di natura completamente diversa (e.g. problemi gestionali).

MODELLO DI VON NEUMANN: CARATTERISTICHE

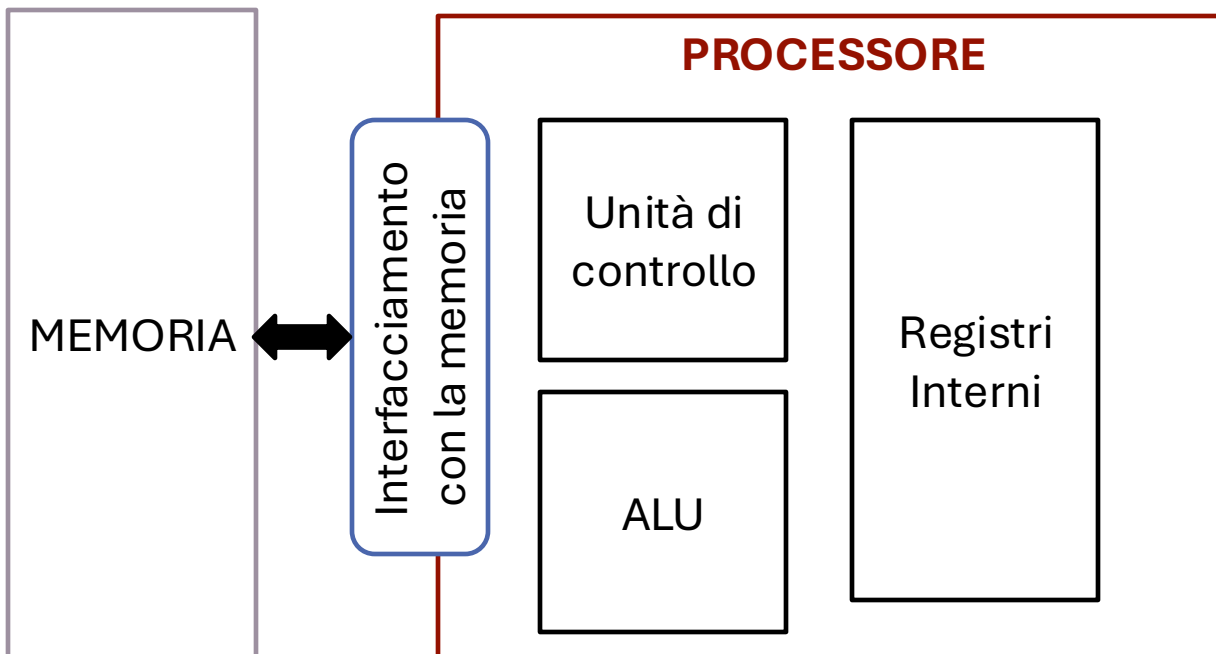
- **Velocità** nell'esecuzione degli algoritmi: milioni di istruzioni svolte dall'unità di controllo in un secondo (MIPS)
- **Affidabilità**
 - Un calcolatore non commette mai errori di algoritmo poiché è un esecutore obbediente dell'algoritmo stesso la cui esecuzione gli è stata affidata
- Adeguata **capacità di memoria** (numero di informazioni)
 - Varia in base al tipo di memoria usato, all'architettura della memoria stessa ed al tipo di informazione
 - Si misura in numero di byte
- **Costo vantaggioso**

IL CALCOLATORE ELETTRONICO

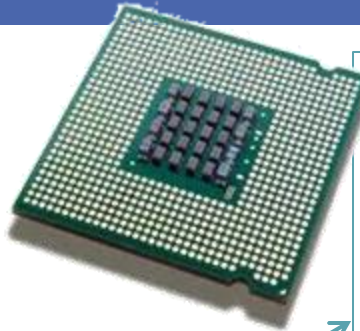
- L'architettura dei calcolatori elettronici è nata ispirandosi al modello generale di Von Neumann.
- Un **calcolatore elettronico** è costituito da componenti elettronici e la sua capacità elaborativa risiede nel **processore**, detto anche **Central Processing Unit**, in grado di eseguire un set di azioni elaborative elementari più o meno complesse (le istruzioni). Una CPU è costituita da tre elementi fondamentali:
 - Unità di controllo (CU) – in senso stretto, talvolta quando ci si riferisce al processore si intende la CU
 - Unità aritmetico-logica (ALU)
 - Registri interni
- Le istruzioni che compongono un programma sono comandi espliciti che:
 - Governano il trasferimento di informazioni sia all'interno del calcolatore, sia tra il calcolatore e i dispositivi di I/O
 - Specificano le operazioni aritmetiche e logiche che devono essere effettuate
- I dati di ingresso e di uscita dell'elaborazioni, nonché la stessa sequenza di istruzioni sono immagazzinati nella memoria centrale
- Il processore preleva ed esegue dalla memoria una ad una.

IL PROCESSORE: UN PRIMO SCHEMA ARCHITETTURALE

- Partendo da un primo schema semplificato in figura, la CPU è composta da:
 - **ALU**: unità logica aritmetica
 - **CU**: unità di controllo
 - Insieme di **registri interni**



IL PROCESSORE: CONTROL UNIT (CU)

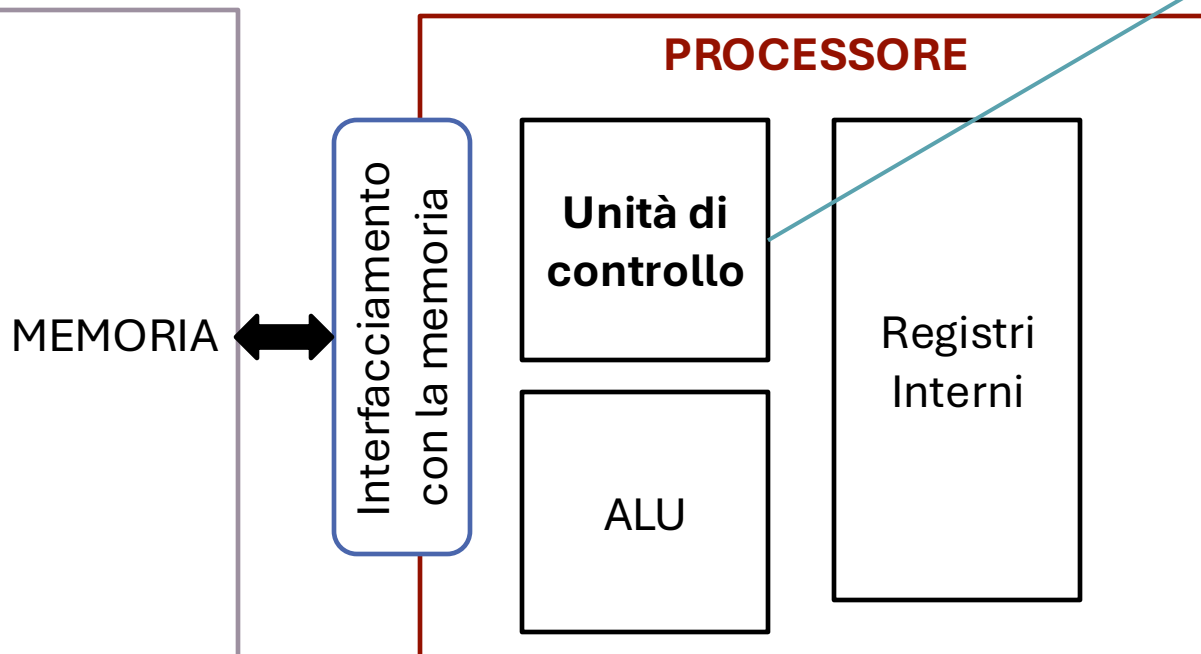


L'**unità di controllo** interpreta le singole istruzioni e attiva tutti i meccanismi necessarie per eseguirle. Ha il compito di:

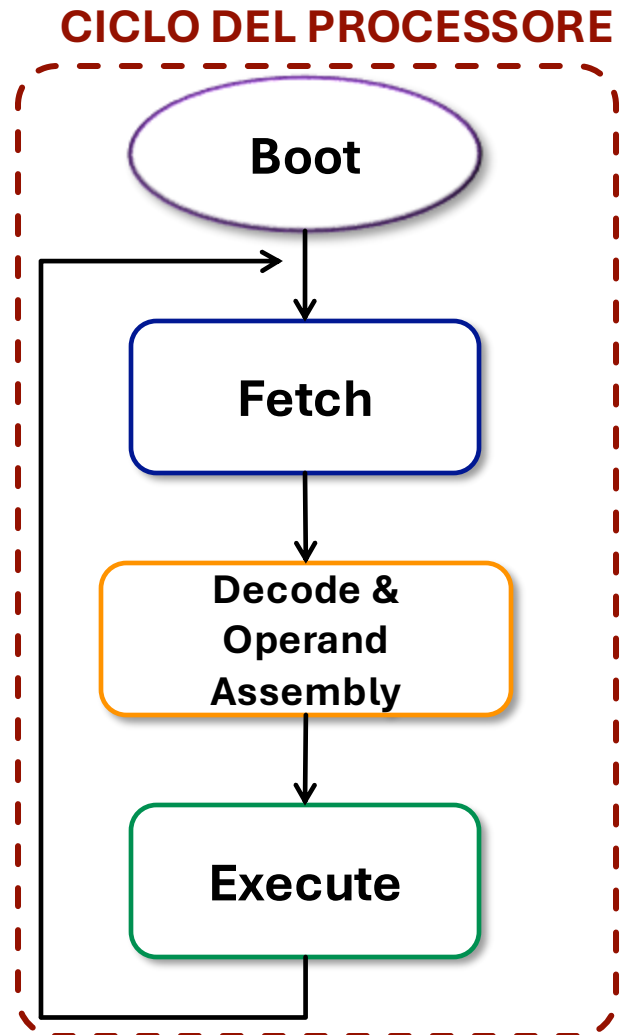
1. **prelevare** ogni istruzione dalla memoria centrale;
2. **decodificare** l'istruzione per capire che operazione fare, quali dati utilizzare ed eventualmente **prelevare** gli **operandi** dalla memoria
3. **eseguire** l'istruzione in base al **codice operativo**.

Ad esempio se l'istruzione prelevata è di tipo aritmetico e richiede due operandi, la CU predispone dapprima il prelievo di tali operandi (eventualmente dalla memoria), attiva poi l'ALU perché esegua l'operazione desiderata, ed infine deposita il risultato di nuovo in memoria.

Al termine dell'esecuzione di una istruzione, la CU procede al prelievo dalla memoria della **successiva istruzione**.



IL PROCESSORE: FASI DEL CICLO FONDAMENTALE DEL PROCESSORE



L'**unità di controllo** interpreta le singole istruzioni e attiva tutti i meccanismi necessarie per eseguirle. Ha il compito di:

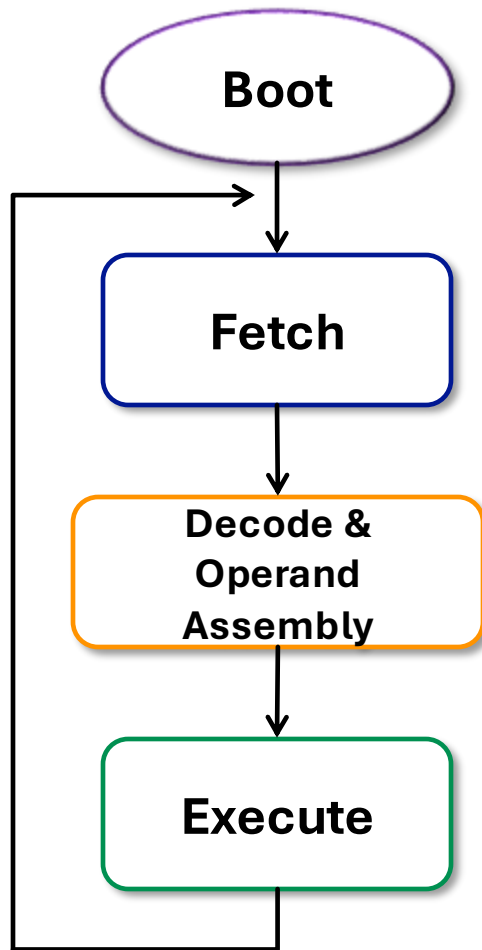
1. **prelevare** ogni istruzione dalla memoria centrale;
2. **decodificare** l'istruzione per capire che operazione fare, quali dati utilizzare ed eventualmente **prelevare** gli **operandi** dalla memoria
3. **eseguire** l'istruzione in base al **codice operativo**.

Ad esempio se l'istruzione prelevata è di tipo aritmetico e richiede due operandi, la CU predispone dapprima il prelievo di tali operandi (eventualmente dalla memoria), attiva poi l'ALU perché esegua l'operazione desiderata, ed infine deposita il risultato di nuovo in memoria.

Al termine dell'esecuzione di una istruzione, la CU procede al prelievo dalla memoria della **successiva istruzione**.

IL PROCESSORE: FASI DEL CICLO FONDAMENTALE DEL PROCESSORE

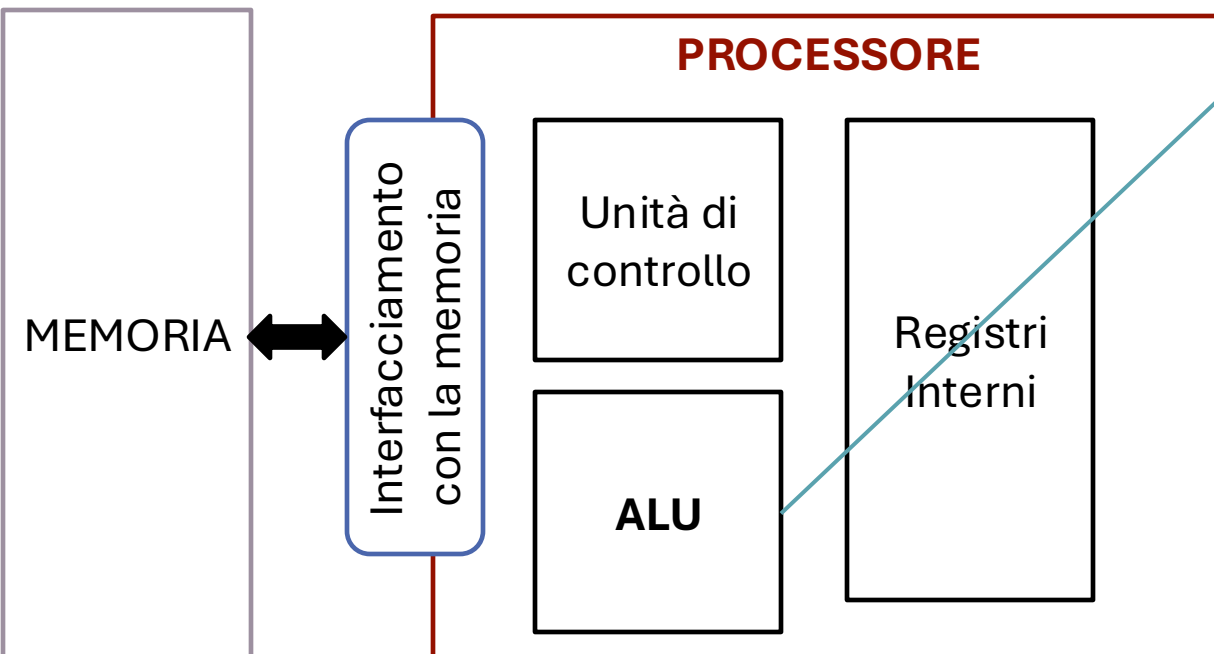
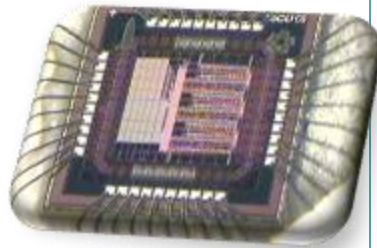
CICLO DEL PROCESSORE



- **Boot:** fase iniziale atta ad informare la CU dell'indirizzo della locazione di memoria che contiene la **prima istruzione** da eseguire
- **Fetch** (prelievo): un'istruzione viene prelevata dalla memoria centrale e memorizzato in un registro interno del processore
- **Decode & Operand Assembly** (decodifica e preleva operandi): interpreta il codice operativo (che indica l'operazione da eseguire) dell'istruzione da eseguire e preleva gli operandi dalla memoria (se servono all'istruzione).
- **Execute:** l'istruzione viene eseguita ed il risultato viene eventualmente salvato.

Nota: nel ciclo fondamentale, ogni istruzione deve essere portata a termine prima di prelevarne un'altra. Vedremo poi come questo ciclo viene modificato per poter gestire quelle che saranno chiamate *interruzioni*.

IL PROCESSORE: ARITHMETIC-LOGIC UNIT (ALU)



L'**unità aritmetico-logica** esegue operazioni aritmetiche, logiche e bitwise (cioè bit a bit) su operandi che le vengono forniti dalla CU (eventualmente prelevati dalla memoria).

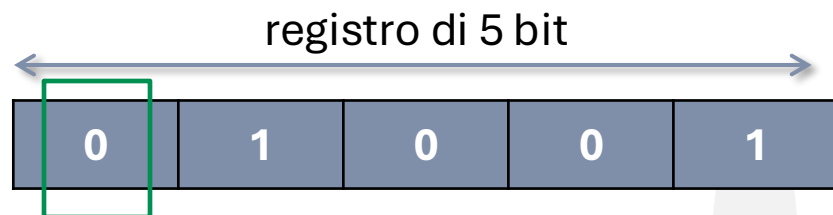
La specifica operazione da eseguire, che dipende dal codice operativo, viene abilitata/selezionata dalla CU tramite appositi segnali di controllo.

L'esito dei suoi calcoli viene segnalato in appositi bit in un registro di stato, chiamati **Condition Code**.

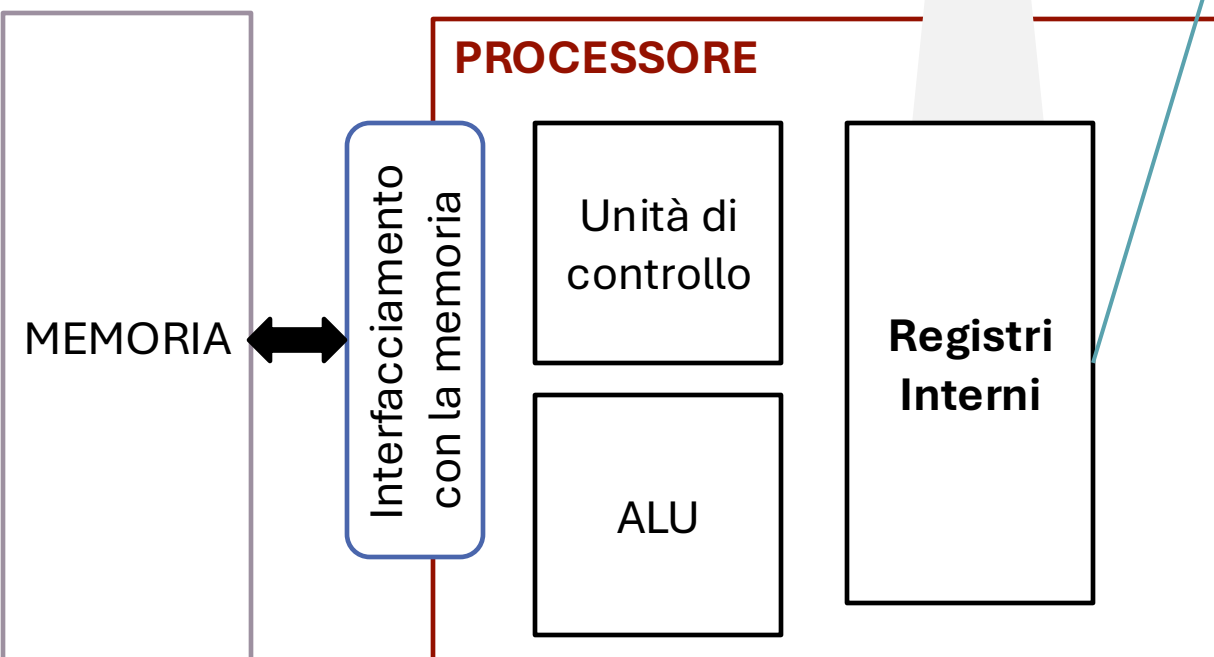
- Questi bit segnalano eventi notevoli, come l'ottenimento di un risultato nullo o negativo e la generazione di un riporto o di un overflow.

A seconda dei processori l'ALU può essere molto complessa. Nei sistemi attuali, l'ALU viene affiancata da processori dedicati alle operazioni sui numeri in virgola mobile, detti *processori matematici*.

IL PROCESSORE: REGISTRI INTERNI



Bit memorizzato in un FF



Un **registro interno** del processore è un elemento di memoria apposito, caratterizzato da velocità di lettura e scrittura elevatissima.

- Sono tipicamente individuati da un «nome» o identificativo, mentre le locazioni/registri di memoria (esterne al processore!!) sono individuati tramite un indirizzo.
- Ogni registro è in grado di memorizzare un numero n di bit mediante n flip-flop.

Il numero e tipo di tali registri varia a seconda dell'architettura della CPU.

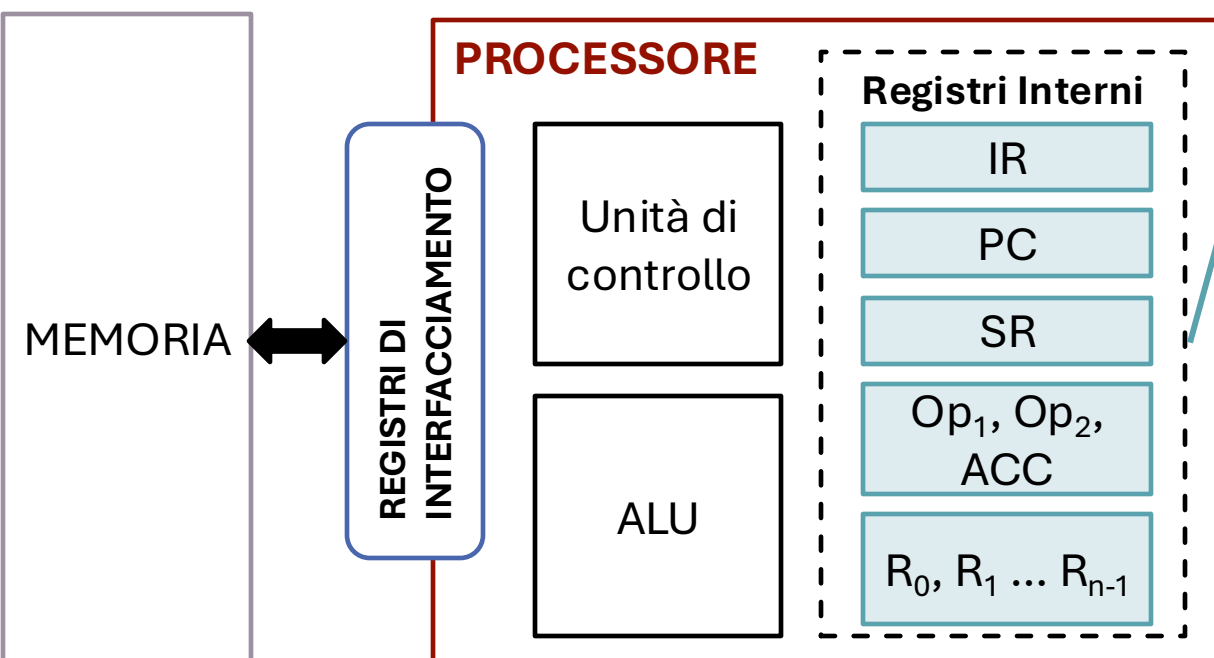
La lunghezza in bit dei registri della CPU viene spesso usata per indicare il «*numero dei bit*» del processore:

- Processori a 8 bit (anni '70/'80) – es. Intel 8080, Motorola 6809
- Processori a 16 bit (anni '80) – es. Intel 8086, Motorola 68000 (16/32 bit)
- Processori a 32 bit (anni '90-inizio anni 2000) – es Intel 80386, Motorola 68030
- Processori a 64 bit (anni 2000)

IL PROCESSORE: PRINCIPALI REGISTRI INTERNI

Questi sono anche detti **registri ad uso speciale** del processore, perché a differenza dei registri generali, sono utilizzati per specifiche funzioni.

Tra questi rientrano **anche i registri di interfacciamento con la memoria (MAR ed MDR)** che vediamo qualche slide più avanti.



- **IR (Instruction Register):** contiene l'**istruzione corrente in esecuzione**, cioè prelevata precedentemente dalla memoria e che la CU sta eseguendo.

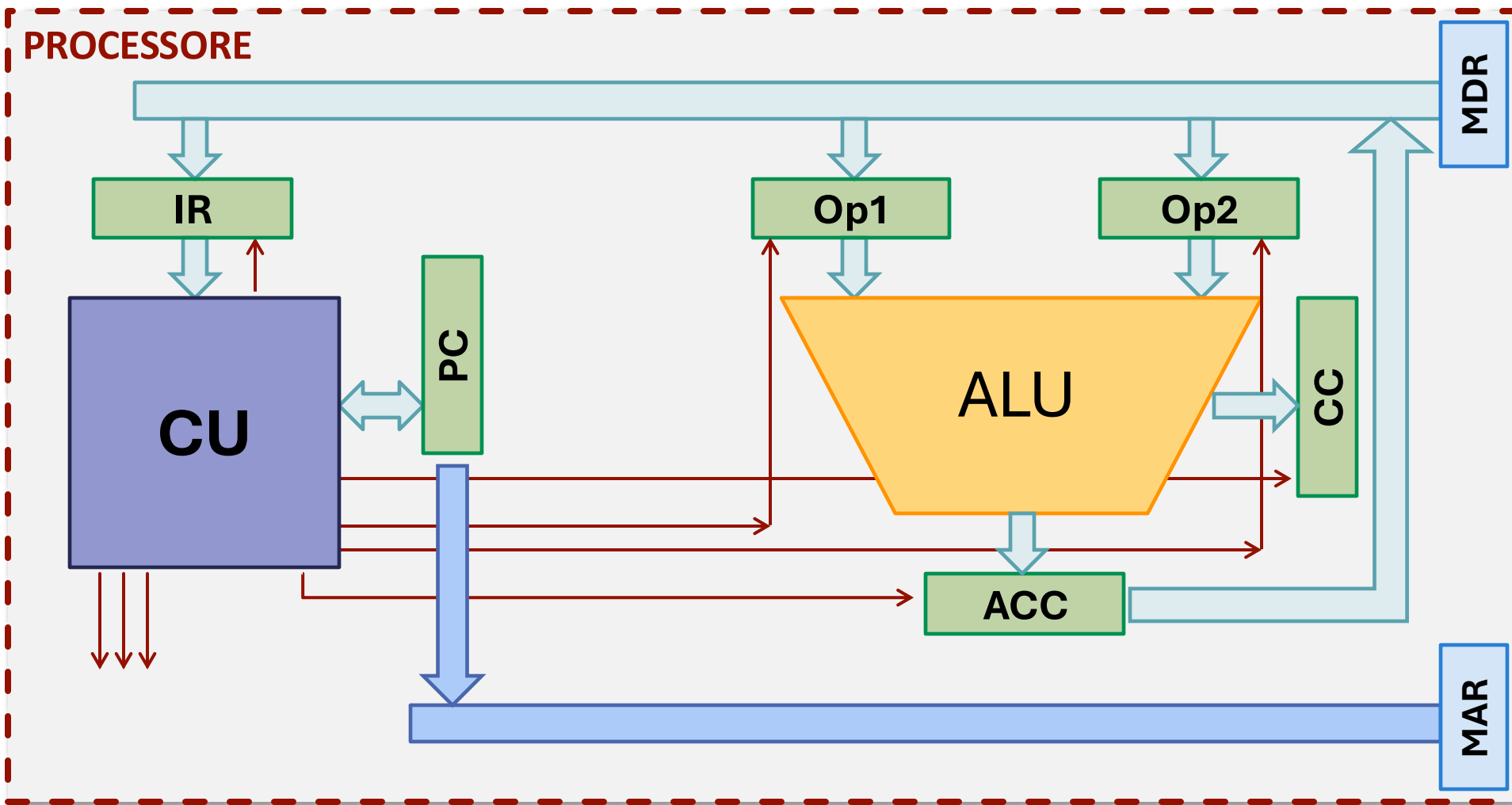
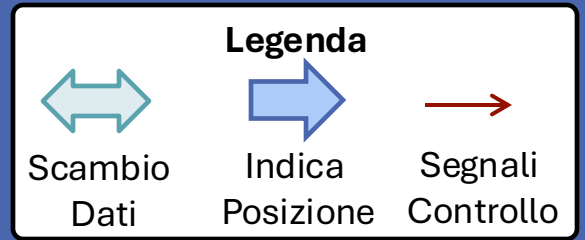
PC (Program Counter): indica la **posizione** in memoria (indirizzo) della **prossima istruzione** da eseguire. Dopo ogni prelievo di una istruzione dalla memoria il suo valore viene aggiornato in maniera tale da puntare alla prossima istruzione.

- **ACC (Accumulatore), Op₁ e Op₂:** sono tre registri diversi e servono come **deposito** di dati da parte dell'**ALU**, tipicamente op₁ e op₂ contengono gli operandi dell'operazione da eseguire e ACC il risultato calcolato al termine dell'operazione.

- **SR (Status Register):** contiene i CC (codici di condizione del processore) che definiscono lo stato del processore, ad esempio in relazione ad un'operazione eseguita dalla ALU.

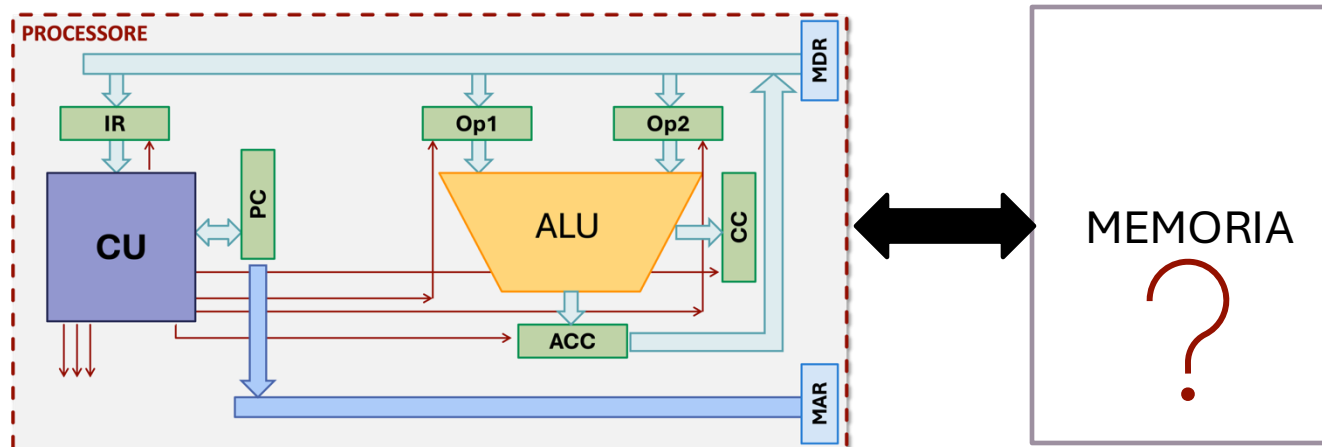
- **R₀, R₁ ... R_{n-1} (Registri Generali):** servono principalmente come deposito di dati di uso immediato.

DETTAGLIO DI ARCHITETTURA INTERNA DEL PROCESSORE

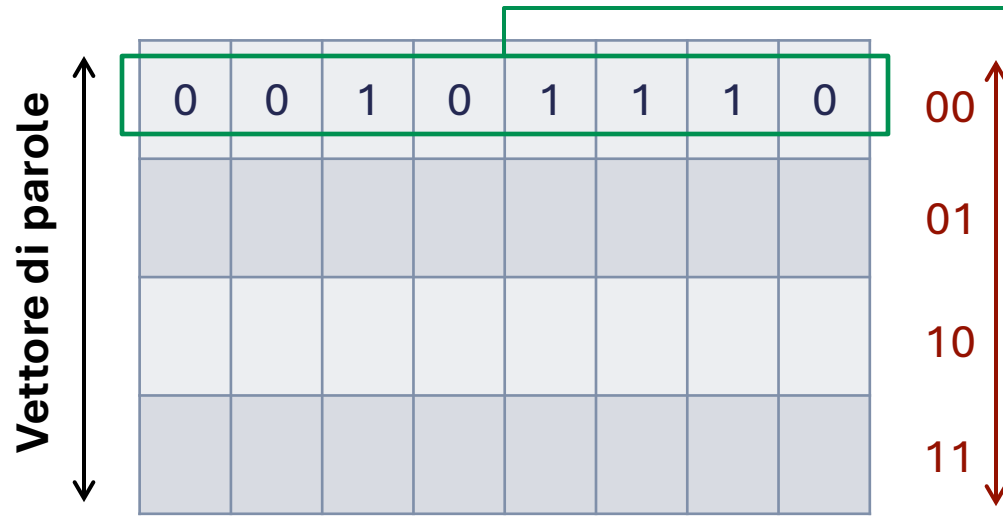


LA MEMORIA CENTRALE

- La memoria centrale di un calcolatore è organizzata come un array di stringhe di bit di lunghezza m , dette **locazioni** o parole. Il valore m è detto **lunghezza di parola**.
- Gli m bit di una locazione sono accessibili al processore (in lettura/scrittura) mediante **un'unica operazione**.
- Ogni parola è individuata da un **indirizzo**, cioè un intero compreso in $[0, N-1]$ con $N=2^n$ il numero di bit usati per codificare gli indirizzi. L'intervallo $[0, N-1]$ prende il nome di **spazio di indirizzamento**.
 - La corrispondenza tra indirizzo e parola è stabilita assegnando indirizzi consecutivi a parole consecutive partendo dall'indirizzo 0.
- La memoria centrale è ad **accesso casuale** (RAM), cioè il tempo di accesso non dipende dalla posizione del dato.



LA MEMORIA CENTRALE: ESEMPIO



**Spazio di
indirizzamento**
[0,3]

locazione/parola
= stringa di $m(=8)$
bit



MSB
bit7

LSB
bit0

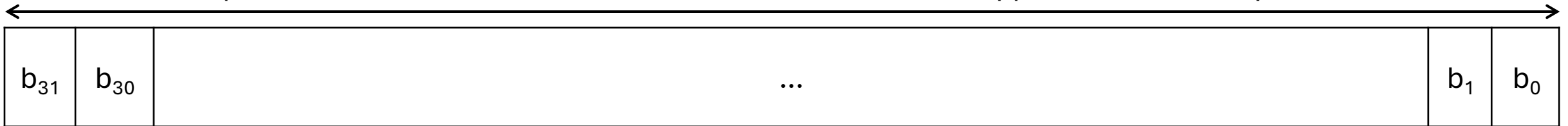
La memoria è composta da 4 locazioni, ciascuna da 8 bit. Quindi la lunghezza di parola m è 8, mentre la lunghezza in bit dell'indirizzo n è 2. Lo **spazio di indirizzamento** è $[0, 2^n - 1]$. $N = 2^n = 4$ è il **numero di parole**, locazioni o celle della memoria.

La locazione selezionata in verde è individuata tramite **l'indirizzo 00**.

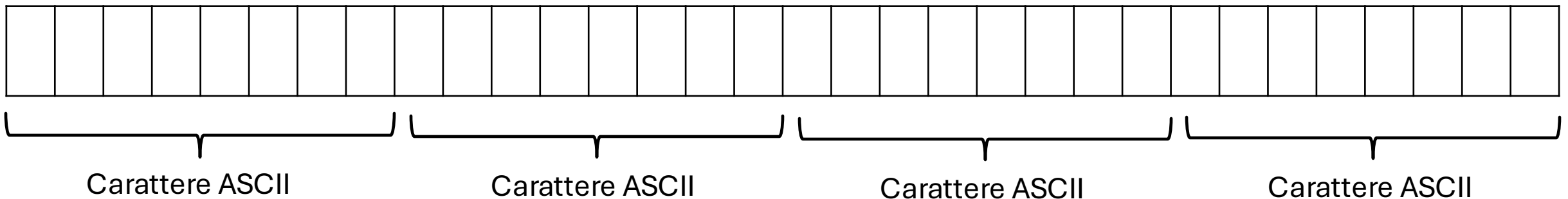
LA MEMORIA CENTRALE

- I calcolatori odierni hanno dimensione di parola variabile a seconda del modello, tuttavia per lo più si usano lunghezze comprese nell'intervallo da 16 a 64 bit.
- Ad esempio, un calcolatore caratterizzato da una lunghezza di parola pari a 32 bit può contenere in una sola parola di memoria un numero naturale (o un numero in complementi a due) da $n = 32$ bit, oppure 4 caratteri ASCII, ciascuno da 8 byte.

parola di memoria di 32 bit contenente un intero relativo rappresentato in complementi a 2



oppure



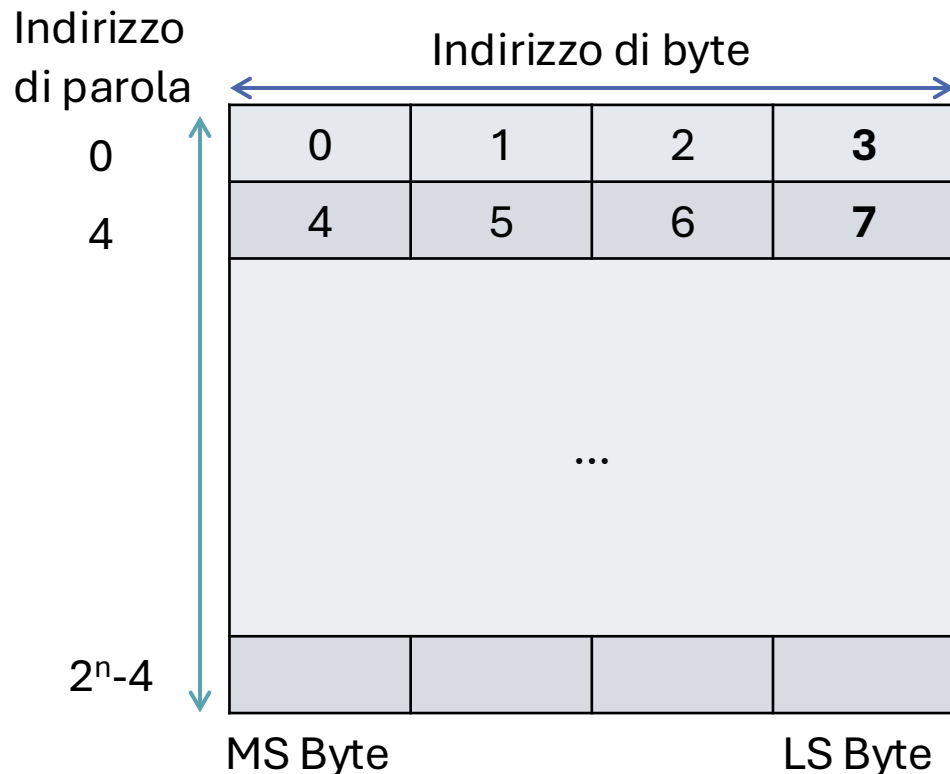
ORGANIZZAZIONE DELLA MEMORIA

- Un calcolatore lavora su tre elementi base di informazione: il bit, il byte (8 bit) e la parola che varia da 16 a 64 bit a seconda del modello di calcolatore. Sarebbe poco pratico assegnare un indirizzo a ciascun bit della memoria!
 - Pressoché tutti i calcolatori moderni adottano il **byte** come **unità minima di informazione indirizzabile** in memoria: si dice che la memoria è indirizzabile per byte (**byte-addressable**).
 - A byte consecutivi si assegnano indirizzi consecutivi (0,1,2,3..) e ne consegue che la parola di memoria, che è formata da un numero intero di byte, deve avere un *indirizzo multiplo della rispettiva lunghezza in byte*. Per esempio se la parola è di 32 bit, ovvero 4 byte, a parole consecutive corrispondono indirizzi multipli di 4, vale a dire 0, 4, 8, 12 ...
 - Il numero di bit che possono essere prelevati dalla memoria o inseriti in memoria simultaneamente dal processore si chiama **parallelismo**: come detto in precedenza, i sistemi moderni accedono alla memoria con parallelismo di parole da 16, 32, o 64 bit. In questo caso sono detti «*processori a parola*».
- I byte costituenti una locazione o parola possono essere disposti in due modi alternativi, denominati **big endian** e **little endian**.

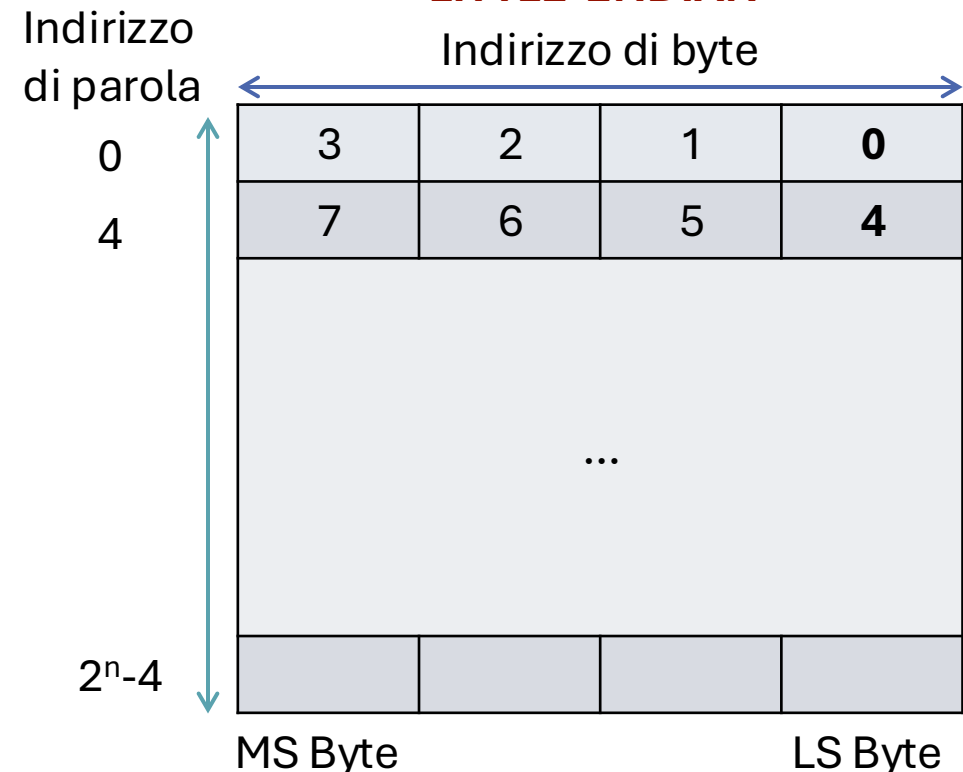
ORGANIZZAZIONE DELLA MEMORIA

- Nell'indirizzamento **big-endian** (**crescente**) l'indirizzo aumenta al diminuire del peso aritmetico del byte, ovvero spostandosi verso il bit meno significativo.
- Nell'ordinamento **little-endian** (**decrescente**) il byte più significativo della parola ha l'indirizzo più alto.

BIG-ENDIAN

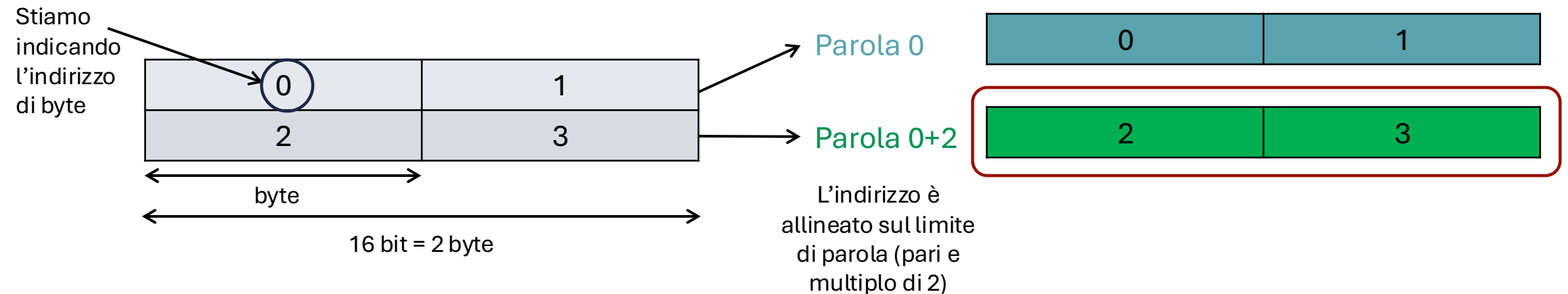


LITTLE-ENDIAN



ALLINEAMENTO DI PAROLA

- Quando la transizione tra una parola e quella consecutiva cade ad un indirizzo multiplo della lunghezza di parola misurata in byte, si dice che lo schema di indirizzamento di parola è di tipo **allineato**.
 - Se ad esempio la parola è da 16 bit, lo schema di indirizzamento di parola allineato prevede che gli indirizzi di parola siano 0, 2, 4, 6 e così via. Se fosse da 64 bit = 8 byte, dovrebbero essere 0, 8, 16, 24 e così via.
 - Esempio per processore a 16 bit:**

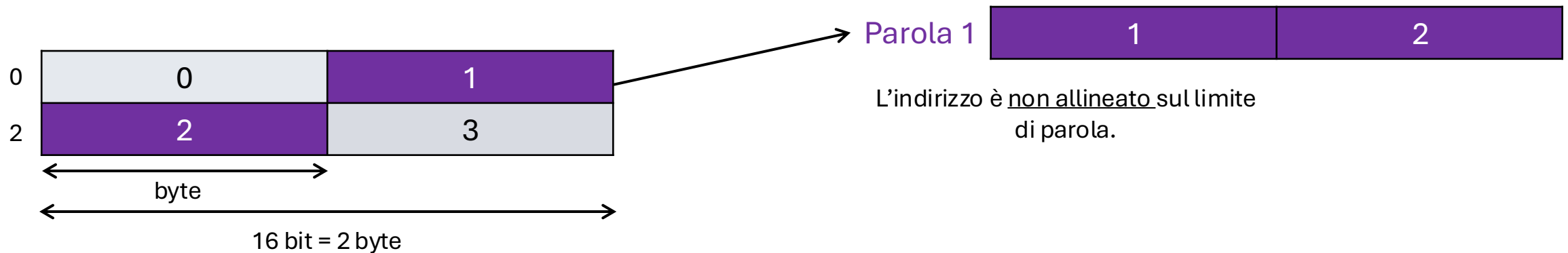


- Un processore di questo tipo è in grado di accedere ai due byte (e.g. 2 e 3) che costituiscono la parola allineata mediante una sola operazione di lettura.

ALLINEAMENTO DI PAROLA

- Alcuni processori consentono l'utilizzo di parole **non allineate** (cioè parole che iniziano ad indirizzo dispari). In tal caso la parola inizia ad un byte con indirizzo scelto in modo arbitrario.

- Esempio per processore a 16 bit:**

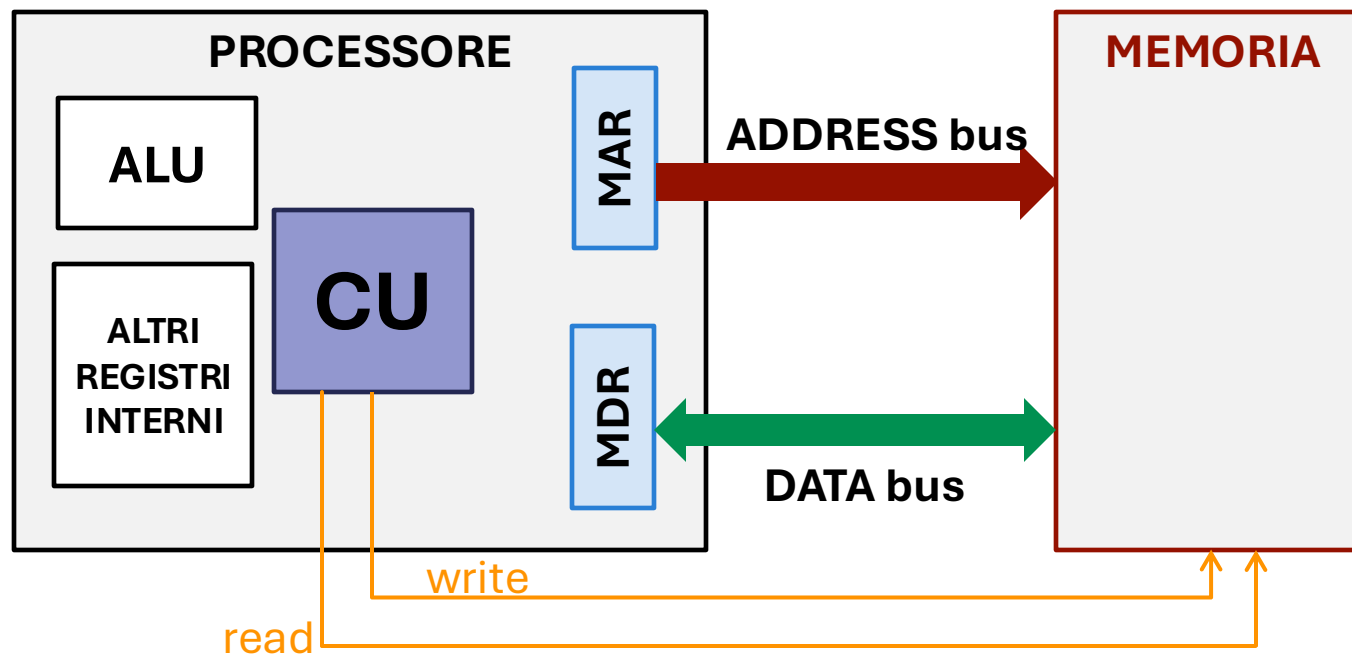


OPERAZIONI DI MEMORIA

- Durante l'esecuzione sia le istruzioni componenti il programma sia i dati su cui esse lavorano, sono contenuti in memoria codificati sotto forma di una o più parole. L'UC del processore deve pilotare il trasferimento dalla memoria al processore o viceversa. Le due operazioni di base della memoria sono **caricamento (load)** e **memorizzazione (store)**.
- Nell'operazione di **load**, **l'UC prende il dato contenuto nella parola di memoria** collocata ad un indirizzo scelto in modo opportuno e lo scrive all'interno del processore; il contenuto della parola di memoria resta inalterato.
 - Per iniziare un'operazione di load, il processore deve inviare alla memoria l'indirizzo della parola interessata ed i comandi che attivano la lettura del contenuto.
 - La memoria reagisce mandando al processore il dato contenuto nella parola indirizzata.
- Nell'operazione di **store**, **l'UC manda il dato alla parola di memoria** collocata ad un indirizzo scelto in modo opportuno e lo scrive all'interno della parola; il contenuto della parola di memoria precedente viene sovrascritto.
 - Il processore manda alla memoria l'indirizzo della parola interessata, il dato da scrivere ed i comandi che attivano la scrittura del contenuto di memoria.
 - La memoria reagisce sostituendo il nuovo dato a quello precedentemente contenuto nella parola indirizzata.

INTERAZIONE PROCESSORE-MEMORIA

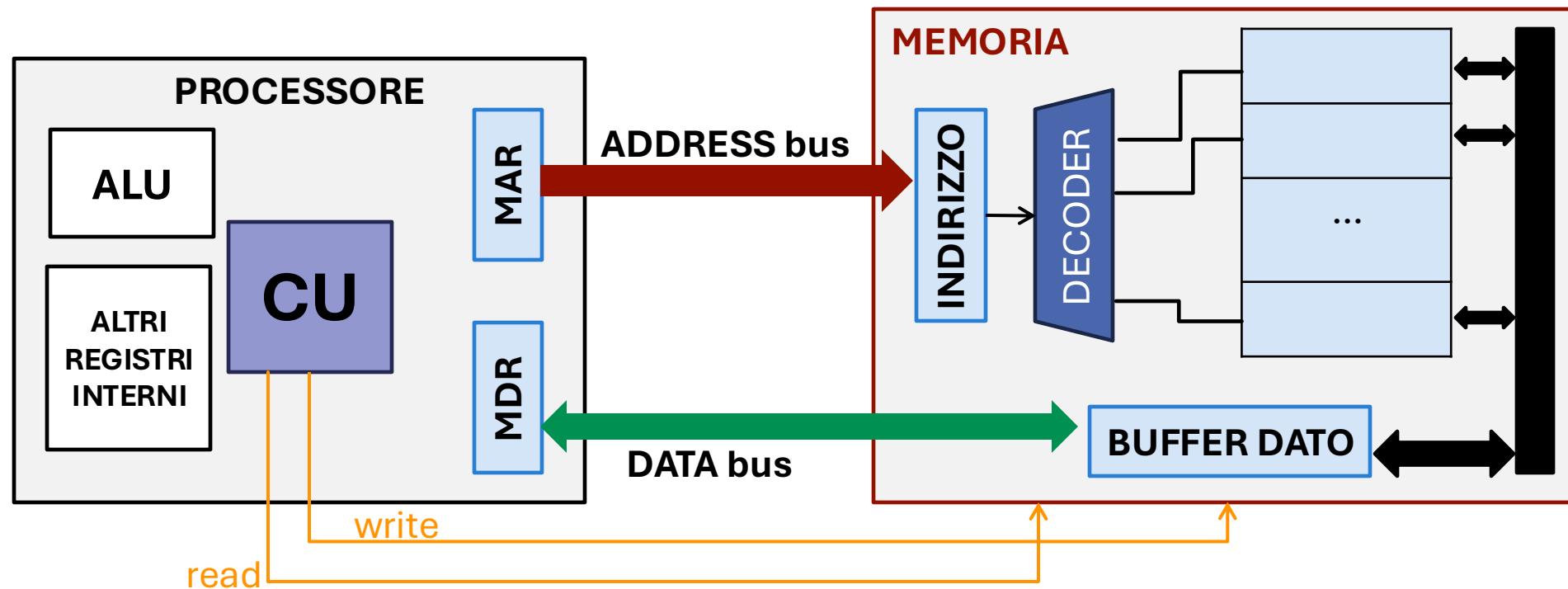
- L'interfaccia tra processore e memoria prevede due registri:
 - **MAR – Memory Address Register:** registro interno del processore atto a contenere l'indirizzo della locazione di memoria a cui fare accesso.
 - **MDR – Memory Data Register:** registro interno del processore atto a contenere il dato da inserire o prelevare dalla locazione di memoria.



La CU provvede a fornire l'indirizzo sul **bus indirizzi**, mentre i dati viaggiano su un **bus dati** separato. La CU genera i **segnali di controllo read e write**.

INTERAZIONE PROCESSORE-MEMORIA

- La CU **indica l'indirizzo** del registro interessato dall'operazione sul bus indirizzi.
- La memoria **decodifica** l'indirizzo abilitando solo il registro ad esso corrispondente affinché:
 - Per una operazione STORE, il dato del buffer sia copiato nel registro
 - Per una operazione LOAD, il dato del registro sia copiato nel buffer.



CICLO DEL PROCESSORE DETTAGLIATO RISPETTO AI REGISTRI INTERNI

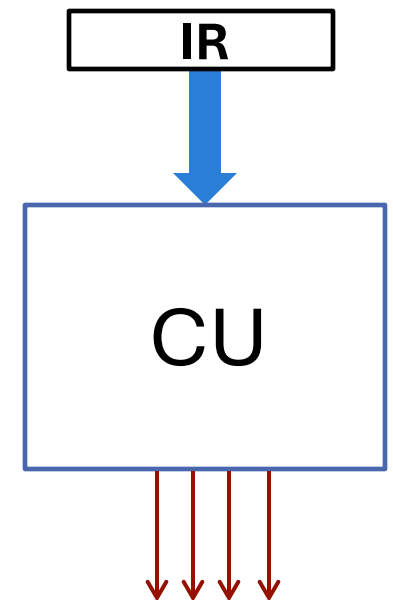
- La **fase di fetch** prevede il prelievo dalla memoria dell'istruzione da eseguire e il calcolo dell'indirizzo della prossima istruzione.
 1. La CU inserisce nel MAR l'indirizzo dell'istruzione da prelevare, presente nel PC.
 2. La risposta della memoria viene depositata nel registro IR.
 3. La CU calcola l'indirizzo della prossima istruzione (utilizzando il contenuto di IR) e aggiorna il PC.
- La **fase decode&operand assembly** serve alla CU per predisporre gli operandi che servono all'esecuzione dell'operazione indicata dal codice operativo (indicato nell'istruzione).
 1. La CU identifica i puntatori ai dati di input e output dell'istruzione ed in caso di operandi di input presenti in memoria, ne effettua il prelievo.
- La **fase execute** consiste nel compiere le azioni richieste dal codice operativo presente nel registro IR.
 1. La CU genera i segnali di controllo necessari per eseguire l'istruzione. Ad esempio, nel caso di un'istruzione di addizione attiva la porzione dell'ALU preposta all'esecuzione di addizioni.
 2. Se l'istruzione lo prevede, vengono settati i codici di condizione della ALU.
 3. Nel caso vengano prodotti risultati, la CU provvede a generare i segnali di controllo per la loro memorizzazione nei registri interni del processore oppure in memoria (utilizzando i puntatori ai dati di output identificati nella fase precedente).

TIPI DI ISTRUZIONI

- Le istruzioni sono operazioni semplici:
 - Trasferimento dati:
 - Da un registro ad un altro registro
 - Da memoria a memoria
 - Da memoria a registri della CPU o viceversa
 - Da memoria ad output, da input a memoria
 - Operazioni aritmetiche o logiche eseguite dall'ALU
 - Verifica di condizioni e confronti
 - Salto condizionato o incondizionato:
 - le istruzioni di un programma sono normalmente prelevate da locazioni consecutive ed eseguite una dopo l'altra, tuttavia può capitare che ci siano istruzioni di salto che modificano l'ordine strettamente sequenziale. In tal caso l'UC determina la nuova istruzione da caricare durante l'esecuzione dell'istruzione di salto.
 - Collegamento a sottoprogramma:
 - sono istruzioni di salto che implementano meccanismi necessari a consentire il ritorno al programma chiamante (salvataggio e ripristino dell'indirizzo dell'istruzione successiva al salto ad un sottoprogramma)
 - Input/Output

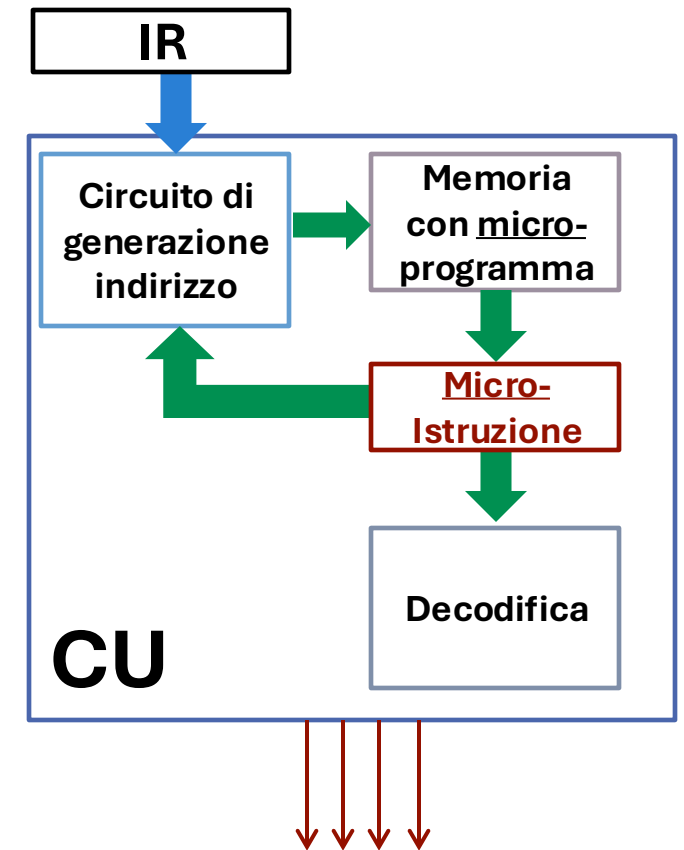
LOGICA CABLATA E MICROPROGRAMMATA

- In generale, una unità di controllo è un automa che ripete senza sosta l'esecuzione del ciclo fondamentale (fetch, decode, execute). L'esecuzione di un'istruzione da parte dell'UC consiste nella generazione dei segnali di controllo che abilitano gli altri componenti architetturali. L'effetto è l'esecuzione dell'operazione specificata dall'IR.
- Le prime CPU venivano realizzate con circuiti in **logica cablata**: il modo in cui viene eseguita un'istruzione è cablato (collegamenti fisici dell'hardware).
 - Le FSM rappresentano il modo più naturale per implementare l'UC in un sistema digitale: gli stati della FSM rappresentano i diversi stati della UC, le uscite della FSM in ogni stato rappresentano i segnali di controllo forniti alle altre unità che eseguono le operazioni.
 - **La UC viene realizzata con logica combinatoria specificamente progettata per pilotare i diversi segnali di controllo nelle diverse «fasi» di elaborazione** ed eseguire operazioni complesse come la lettura di un dato in memoria, la sua modifica e il suo salvataggio direttamente in memoria tramite una singola istruzione.
- Concettualmente semplice e veloce, ma costoso e difficile da modificare ed estendere



LOGICA CABLATA E MICROPROGRAMMATA

- Al contrario, nella **logica microprogrammata** esiste un circuito interno all'UC che provvede alla generazione degli indirizzi delle **microistruzioni**.
- Le microistruzioni sono i micropassi in cui è possibile dividere un'istruzione complessa e sono salvate in una memoria detta **microROM**.
- Una volta selezionata la microistruzione, il circuito di decodifica viene utilizzato per generare i segnali di abilitazione da attivare in un singolo passo di elaborazione.
- Dal punto di vista realizzativo è più complesso e più lento per via degli accessi alla memoria, ma è maggiormente più flessibile



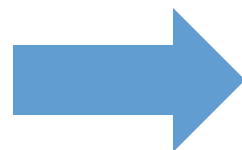
LINGUAGGIO MACCHINA

- Il **linguaggio macchina** è quello compreso dal processore. Se da un lato il linguaggio macchina ha il vantaggio di alta velocità di esecuzione e ottimizzazione dell'uso delle risorse hardware, dall'altro ha lo svantaggio di **non** permettere la **trasportabilità** dei programmi tra processori differenti.
 - Programmare in linguaggio macchina non è facile perché le istruzioni esprimono funzionalità elementari, ma la realizzazione di algoritmi richiede un gran numero di comandi che sono espressi come sequenze di bit.
- **Un'istruzione in linguaggio macchina** è *sul piano astratto* una tripla strutturata $i = (COP, OS, OD)$ dove:
 - $COP \in \mathcal{C}$ insieme dei **codici operativi** del processore, cioè delle operazioni elementari definite al livello del linguaggio macchina;
 - OS è un insieme di **operandi sorgente**, cioè di valori e/o puntatori a registri (interni del processore oppure registri di memoria) contenenti i valori su cui opera f.
 - OD è l'insieme di **operandi destinazione**, cioè di puntatori a registri (interni del processore oppure registri di memoria) cui sono destinati i risultati dell'elaborazione f.

ESEMPIO

$i_1 = (C_{op1}, OS_1, OD_1)$
 $i_2 = (C_{op2}, OS_2, OD_2)$
 $i_3 = (C_{op3}, OS_3, OD_3)$
 $i_4 = (C_{op4}, OS_4, OD_4)$
 $i_5 = (C_{op5}, OS_5, OD_5)$

Programma



3000
3004
3008
3012
3016
3020
3024
3028
3032
3036
3040

$i_2 = (C_{op2}, OS_2, OD_2), 3024$
$i_1 = (C_{op1}, OS_1, OD_1), 3004$
$i_3 = (C_{op3}, OS_3, OD_3), 3028$
$i_4 = (C_{op4}, OS_4, OD_4), 3032$
$i_5 = (C_{op5}, OS_5, OD_5), XXXX$

**Programma allocato in
memoria**

Questo è un esempio in cui le istruzioni non sono memorizzate in maniera consecutiva in memoria. Ai fini dell'esempio, in ciascuna locazione è indicata l'istruzione e **l'indirizzo della prossima istruzione**. Successivamente vedremo come la CU gestisce i salti e il calcolo del next address.

**Indirizzo della
prima istruzione
da eseguire
caricato in fase
di boot**

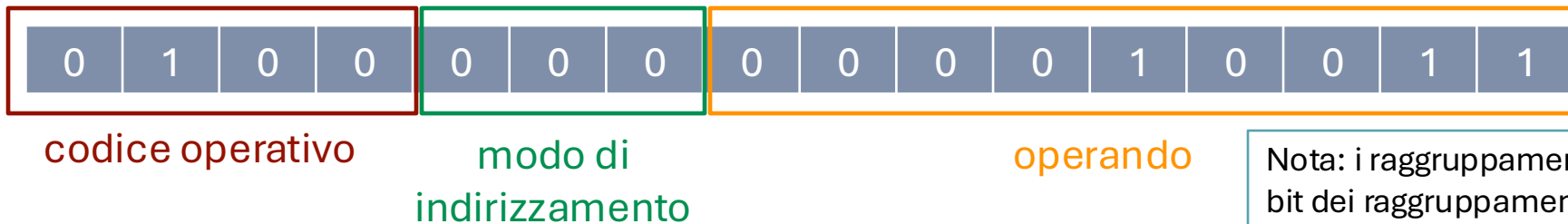
CODIFICA DI UN'ISTRUZIONE MACCHINA

- Una istruzione è codificata mediante una stringa di bit che può essere *a lunghezza fissa* (tipicamente 32 bit) o a *lunghezza variabile*
- I bit della codifica sono raggruppati in vari campi, che identificano in maniera univoca il **codice operativo** e gli operandi:
 - tipicamente per un processore sono definiti diversi **formati di istruzione**: ogni formato si applica ad un insieme di istruzioni che hanno in comune il numero e tipo di operandi, e definisce la lunghezza e il significato dei vari campi dell'istruzione

OPERANDI DI UN'ISTRUZIONE

- Per poter effettuare l'operand assembly, la CPU deve sapere *come deve accedere agli operandi*.
- Gli operandi di un'istruzione possono essere:
 - implicitamente definiti dall'istruzione
 - specificati direttamente all'interno dell'istruzione stessa come valori costanti (sia dati veri e propri che indirizzi)
 - contenuti in registri interni del processore di cui viene fornito l'identificativo all'interno dell'istruzione
 - contenuti in locazioni di memoria di cui si fornisce un puntatore all'interno dell'istruzione
- I diversi modi possibili di specificare gli operandi di un'istruzione si chiamano **modi di indirizzamento**
- *Ogni processore ammette un set specifico di modi di indirizzamento possibili*
- Ciascuna istruzione ammette un set specifico di modi di indirizzamento applicabili ai propri operandi

Esempio di
istruzione a 16
bit:



Nota: i raggruppamenti, il numero di bit dei raggruppamenti e il significato (COP, indirizzamento, etc) dipendono dallo specifico processore

LINGUAGGIO ASSEMBLY

- Il **linguaggio assembly**, anche detto di basso livello, mantiene uno stretto legame con il linguaggio macchina, rendendo la programmazione a questo livello più agevole, sostituendo alle sequenze di bit dei **codici mnemonici facili da interpretare e ricordare**.
 - Sono comunque legati all'architettura della macchina!
- Il linguaggio assembly definisce **l'Instruction Set Architecture (ISA)** di un processore
- **La corrispondenza fra una istruzione in linguaggio assembly e una in linguaggio macchina NON è 1:1**
 - Struttura interna diversa (es. diverso ordine degli operandi nell'istruzione)
 - Omissione di alcuni operandi impliciti (specificati invece nell'istruzione in linguaggio macchina)
 - Utilizzo di istruzioni assembly che corrispondono ad una sequenza di istruzioni macchina
 - Utilizzo di istruzioni assembly che non corrispondono a nessuna istruzione macchina (es. definizione di costanti usate nel codice, direttive di allocazione del codice).

PROCESSORI RISC E CISC

- Il tipo e la complessità delle singole istruzioni, e il numero e il tipo di modi di indirizzamento ammessi non sono uguali per tutti i processori
- Alcuni processori (detti **Reduced Instruction Set Computer - RISC**) hanno delle apposite istruzioni per la movimentazione dati da e verso la memoria (sono perciò anche detti LOAD/STORE),
 - mentre effettuano tutte le altre operazioni utilizzando solo i registri del processore, ammettendo un insieme di modi di indirizzamento molto limitato e un set di istruzioni molto semplici, tipicamente a lunghezza fissa.
- Altri processori (detti **Complex Instruction Set Computer - CISC**) hanno istruzioni più complesse, spesso a lunghezza variabile, con molti modi di indirizzamento. Nascono per colmare il gap semantico con i linguaggi di alto livello: rispetto ad un programma scritto per un processore RISC, quello scritto per un CISC ha meno istruzioni e meno accessi in memoria (ma più lenti), garantendo alta produttività agli sviluppatori

PROCESSORI RISC E CISC

ASPETTO ARCHITETTURALE	CISC	RISC
Numero di istruzioni	Molto elevato, istruzioni complesse	Ridotto, istruzioni semplici
Lunghezza istruzioni	Variabile (da pochi a molti byte)	Fissa (tipicamente 32 bit, a volte 16/64)
Modi di indirizzamento	Numerosi e complessi	Pochi e semplici
Accesso alla memoria	Ammesso da molte istruzioni	Soltanto con Load/Store
Cicli per istruzione	Spesso più cicli clock	Idealmente 1 ciclo per istruzione
Decodifica istruzioni	Complessa, spesso tramite microcodice	Semplice e veloce
Hardware / Software	Hardware più complesso, compilatori più semplici	Hardware più semplice, compilatori più sofisticati
Efficienza energetica	Generalmente inferiore	Generalmente superiore
Esempi tipici	x86, Motorola 68000, VAX	ARM, MIPS, RISC-V, SPARC, PowerPC

CONFRONTO TRA PROCESSORI

Architettura	Tipo	Anno introd.	Usata in	Lunghezza istruzioni	Modi di indirizzamento	Modello memoria	Registri principali
x86 / x86-64 (Intel/AMD)	CISC	1978 / 2003	PC, Server, Workstation	Variabile	Numerosi (~10–12)	Non Load/Store puro	~16 registri generali 64 bit
Motorola 68000	CISC	1979	Amiga, Atari ST, Macintosh, Sega Mega Drive	Variabile	Numerosi (~14 modi)	Non Load/Store puro	8 Data (D0–D7) + 8 Address (A0–A7), SR, PC
ARMv6	RISC	2001	RaspberryPi, smartphone sis. embedded	Fissa 32 bit + Thumb 16 bit	Pochi (3–5)	Load/Store puro	16 registri (R0–R15), PC, LR, CPSR
ARMv8-A (AArch64)	RISC	2011	Smartphone, Tablet, Apple Silicon, Server	Fissa 32 bit (A64)	Pochi e semplici (3–5)	Load/Store puro	31 registri generali 64 bit + PC
RISC-V	RISC	2014	Ricerca, microcontrollori, CPU open hardware	Fissa 32 bit (+ compressa 16 bit opz.)	Pochi, modulari	Load/Store puro	31 registri generali + PC